PROJECT 1 – Document Classification

2026/27

Important: It must be used:

a) **The same name we propose for the functions and procedures**
b) **The same datatypes we propose**
c) **The same output messages**

Otherwise, the correction system will not work, and you will not pass the evaluation.

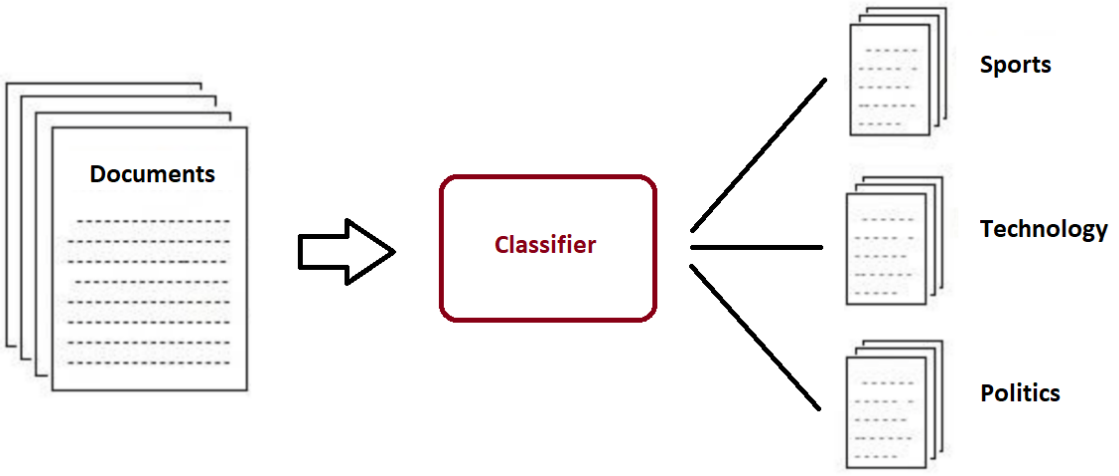
Introduction

Document classification is the process of assigning categories to documents to make them easier to search, sort, analyze, filter, etc. A document contains information related to some specific category (or topics). Assigning one or more classes (categories/topics) to a document makes it extremely beneficial for news sites, blogs, business industries, magazine publishers or anybody dealing with a huge amount of content.

Document classification is seen as a very useful Machine Learning (ML) application in the domain of Natural Language Processing (NLP). In general, there are two types of ML techniques: supervised and unsupervised. A ML model is built using **supervised** methods by creating a **training set**, which contains predefined categories (called **classes**) of documents and every document is tagged with one or more categories (called **labels**). A document **classifier** ML model is then trained on the dataset to perform prediction of a new document's category during the later **test** phase. The classifier may also additionally offer a confidence score to show how confident it is that the classification label is correct, depending on the classification algorithm or strategy utilized.

An example illustration of a Document Classification Model

Intuitively, given that a document is about a particular topic, one would expect some frequent words. For example, "goal" and "corner" will often appear in documents about football, "president" and "ministry" will appear in documents about politics, whereas stop words, like "the" and "with" will appear approximately equally in both. Indeed, words in a document can be used as a powerful tool (called **features**) to help in the prediction of a document category.



Goal:
The project consists of developing a program to classify documents. You are given a collection of news in a **txt** format (`docsNews2026`) about different categories (football, space, health, weather, elections, music and aviation), and a **txt** file with the list of keywords of every category (`keywords_2026.txt`).

Your program must work with any folder of documents and any keywords file with the format described below.

Exercise 1: Implement *DocClass_Welcome()*

Implement a procedure called *DocClass_Welcome()*. This procedure has no input parameters and must display the following informational message:

```
Document Classification is typically used in Document Processing for
assigning categories to documents. Thus, it eases their posterior
processing and analysis.

The program "DocumentClassification" classifies an input folder
containing documents in txt format. It will also receive a file
containing information about the topics to classify.

The program will generate the following files:
A txt file containing the category of the documents
A txt file containing the statistics of the classification
```

Exercise 2: Implement *Read_Categories_Keywords(filename)*

The function *Read_Categories_Keywords()* opens the **txt** file and reads the categories and the keywords of each category. The input is the name of the file containing the lists of categories and keywords (for example, "keywords_2026.txt"), whereas the output are:

- *`l_categories`*: list of categories (list of *strings*).
- *`l_keywords`*: list of keywords per category (list of *sets*). Each element of the list is a *set*, with the keywords of that category.

The input file contains one category per line: the name of the category, a colon (:) and the keywords separated by one or more blank spaces. Lines may end with blank spaces. For example:

```
Category1: KeyWord1A  KeyWord1B  KeyWord1C
Category2: KeyWord2A  KeyWord2B  KeyWord2C  KeyWord2D  KeyWord2E
…
CategoryN: KeyWordNA  KeyWordNB  KeyWordNC  KeyWordND
```

Important: The number of categories and the number of keywords per category are **variable**.

For example, if the list of categories and keywords are:

```
medical: physician infection patient care
space: planet orbit nasa
sports: medal record score team training
```

The output should be:

- `l_categories = ["medical", "space", "sports"]`
- `l_keywords = [{"physician", "infection", "patient", "care"},`
  `{"planet", "orbit", "nasa"},`
  `{"medal", "record", "score", "team", "training"}]`

Exercise 3: Implement *Document_Reader(filename)*

The function *Document_Reader()* receives the path of the file containing the news to read (for example, "`docsNews2026\news_01.txt`" in case of *Windows*). It opens the file, reads its contents and returns the list of words `l_words` (list of *strings*) with the words of the document, in the order in which they appear.

Important:

- This function is **case insensitive**, which means that lower and uppercase letters are equivalent (NASA = nasa, Federer = federer).
- A word is a sequence of **letters and/or digits**. Numbers are words, for example `19`, and so are words mixing letters and digits, like `rocket23`.
- **Any other character is a word separator**: blank spaces, tabs, line breaks, apostrophes (' '), full stops (.), commas (,), question and exclamation marks (? !), quotation marks (" " "), hyphens (-), dashes (– —), colons (:), semicolons (;), parentheses ( ), brackets [ ] { }, ellipsis (…), slashes (/ \), percent sign (%), dollar sign ($), at sign (@), etc.
- The documents are not all saved with the same text encoding: most of them are UTF-8, but some are Windows-1252. Your function must be able to read both (see the Hints section).

For example, if a filename contains the text:

```
"Once rocket23 reached the planet, the NASA-team celebrated a party! My
15-year-old daughter couldn't (barely) sleep; she loves planets."
```

`l_words` will be:

```
["once", "rocket23", "reached", "the", "planet", "the", "nasa", "team",
"celebrated", "a", "party", "my", "15", "year", "old", "daughter",
"couldn", "t", "barely", "sleep", "she", "loves", "planets"]
```

Hint: the string methods `isalnum()`, `lower()` and `split()` are useful here.

Exercise 4: Implement *Count_Frequencies*(*l_words, l_categories, l_keywords*)

Create a function called *Count_Frequencies()* that takes as input the following parameters:

- `l_words`: list of words of the document.
- `l_categories`: list of categories.
- `l_keywords`: list of sets with the keywords of each category.

The function returns:

- `d_frequencies`: dictionary of dictionaries (nested dictionary) with the number of occurrences of each keyword per category. The keys of the outer dictionary are the categories; the keys of each inner dictionary are the keywords of that category, and the values are the number of times the keyword appears in words (0 if it does not appear).

For example, if the lists of categories and keywords are:

```
medical: physician infection patient care
space: planet orbit nasa
sports: medal record score team training
```

and the document file contains the text:

```
"Once rocket23 reached the right distance from the planet, the NASA-team
celebrated a big party at the nasa main building with all teams."
```

`d_frequencies` should be:

```
{'medical': {'physician': 0, 'infection': 0, 'patient': 0, 'care': 0},
'space': {'planet': 1, 'orbit': 0, 'nasa': 2},
'sports': {'medal': 0, 'record': 0, 'score': 0, 'team': 1, 'training':
0}}
```

Note: the word "team" and "teams" are different. Only "team" is a keyword. The order of the keywords inside each inner dictionary is not relevant.

Exercise 5: Implement *Classify_Doc(d_frequencies)***:**

Create a function called *Classify_Doc()* that takes as input the dictionary of frequencies *d_frequencies* and returns:

- `d_votes`: dictionary with the total occurrences and total percentage of each category. The key is the category, and the value is a *tuple* of two *integers*, `(total_occurrences,total_percentage)`.
- `doc_category`: the category of the document (*string*).

For each category, `total_occurrences` is the sum of the occurrences of all its keywords. `total_percentage` is the percentage of these occurrences with respect to the occurrences of the keywords of **all** the categories, rounded to the nearest integer. If no keyword appears in the document, all the percentages are 0 (take care to avoid the division by zero).

For example, if `d_frequencies` is:

```
{'medical': {'physician': 0, 'infection': 0, 'patient': 0, 'care': 0},
'space': {'planet': 1, 'orbit': 0, 'nasa': 2},
'sports': {'medal': 0, 'record': 0, 'score': 0, 'team': 1, 'training':
0}}
```

there are 3 occurrences of `'space'` and 1 of `'sports'` (4 in total), so the output will be:

```
d_votes = {'medical': (0,0), 'space': (3,75), 'sports': (1,25)}
```

The document is assigned the category with the highest percentage, but **only if that percentage is strictly greater than 50%** (compare the exact value, before rounding). Otherwise, the document is classified as `"UNKNOWN"`.

For example, in both of the following cases `doc_category = "UNKNOWN"`:

- `d_votes={'medical':(0,0), 'space':(2,50), 'sports':(2,50)}` (`'space' and 'sports' have both 50%`)
- `d_votes={'medical':(2,40), 'space':(1,20), 'sports':(1,20), 'politics':(1,20)}` (`'medical' has 40%`)

Exercise 6: Implement *main()*:
Implement the main program, which calls the functions and procedures above to classify the documents. The sequence is as follows:

1. Call the procedure `DocClass_Welcome()` to inform about the program.
2. Ask the name of the file that contains the categories and keywords (for example: "`keywords_2026.txt`"). If the file does not exist or contains no data, ask the user again, until a valid file is entered. The file must be in the same directory of your python script.
3. Read the contents of the file, using the function `Read_Categories_Keywords()`.
4. Ask the name of the folder that contains the document to classify (for example, "`docsNews2026`"). If the folder does not exist or contains no txt files, ask the user again, until a valid folder is entered. The folder must be in the same directory of your python script.
5. For each file in the folder, do the following:
   - If it is a txt file, call the functions `Document_Reader()`, `Count_Frequencies()` and `Classify_Doc()`. write in the file `<folder_name>_DocClassification.txt` (for example, `docsNews2026_DocClassification.txt`) the name of the file, its category, the percentage of every category and, unless the category is UNKNOWN, the number of occurrences of each keyword of the assigned category.
   - If it is not a txt file, write the message `Attention: the following non txt file has been detected: <filename>` in the same file.

The format of the output file is:

```
<filename>: <category>(<C1>:<P1>%, <C2>:<P2>%, …): <keyword1>:<N1>,
<keyword2>:<N2>, …
<filename>: UNKNOWN(<C1>:<P1>%, <C2>:<P2>%, …)
Attention: the following non txt file has been detected: <filename>
```

For example (lines shortened with …):

```
news_01.txt: music(football:0%, space:0%, …, music:100%, aviation:0%):
album:0, band:0, …, song:2, songs:0
news_06.txt: UNKNOWN (football:0%, space:0%, health:0%, weather:0%,
elections:0%, music:0%, aviation:0%)
…
Attention: the following non txt file has been detected: notes.docx
```

The use of `os.path.join()` to build the path of the files inside the folder is **mandatory**:

```
file_path = os.path.join(folder_name, file_name)
```

6. Once all files of the folder are classified, write the file `<folder_name>_DCStatistics.txt` (for example, `docsNews2026_DCStatistics.txt`) with:
    - the total number of txt files (non-txt files do not count);
    - the percentage of txt documents of each category, in the order of the keywords file, followed by UNKNOWN, always with **2 decimals**;
    - the **top 10 keywords** of the folder: the keywords (of any category) with more occurrences in all the txt documents, sorted from the most to the least frequent, with their number of occurrences.

    The format of the file is:

```
*** Statistics
Total txt files: N
Category1: XX.XX%
Category2: YY.YY%
…
CategoryN: ZZ.ZZ%
UNKNOWN: WW.WW%

*** Top 10 keywords
KeyWord1: NumOccurrencesKeyWord1
KeyWord2: NumOccurrencesKeyWord2
…
KeyWord10: NumOccurrencesKeyWord10
```

Both output files must be created in the **same folder** as your Python script.

Important Remarks:

- The functions are written before the **main guard**, and the main program is written under it, as follows:

```
import os

def function_n(param_1, …, param_m):
 # the function code goes here
 return out_1, …, out_p

if __name__ == "__main__":
 # the main code goes here
 out_1, …, out_p = function_n(param_1, …, param_m)
```

- Following the Python conventions (PEP 8), the names of variables and parameters are written in lowercase, with words separated by underscores (`doc_category`, `total_occurrences`).
- The management of possible errors (e.g. file does not exist, division my zero) is **mandatory**. For example:

```
try:
    f = open(filename,"r")
except:
    raise FileNotFoundError
```

- Feel free to create more functions if necessary.
- **Comments** in the code are **mandatory**. Add comments to explain your code.
- The program will be evaluated with Python 3.12. Do not use external libraries: only the Python standard library (`os` is enough).

Hints

Some useful instructions for reading all files in a folder. The `os` module in Python provides functions to interact with the operating system. If you import this module:

- `os.listdir(path)` returns the list of all files and folders in the directory `path`.
- `os.path.exists(path)` check if files and/or directories exist. It returns a *Boolean*.
- `os.path.isfile(path)` and `os.path.isdir(path)` check whether a file or a folder exists (they return a Boolean).
- `os.path.join(folder, name)` builds the path of a file inside a folder, on any operating system.

More info at https://www.geeksforgeeks.org/os-module-python-examples/

Text encoding. A file saved with an encoding cannot always be read with another one: reading a Windows-1252 file as UTF-8 raises `UnicodeDecodeError`.
In such a case, try different encodings when opening the file. For example:

- `file = open(filename, "r", encoding="utf-8")`
- `file = open(filename, "r", encoding="windows-1252")`

Always specify the encoding when opening a file: the default one depends on the operating system.

Delivery

The project will be done in pairs, and the delivery will be via *Caronte*. Instructions:

- Upload the file "`DocumentClassification.py`".
- Upload the presentation* in PDF format
- Only **one of the members** of the group must do it.
- At the beginning of this Python file, write, **as comments**, the name, surname and NIU of the members of the group.

Deadline: **Nov 25th, 2026**.

***Presentation**

The project must be presented by the **two members of the group**. Contents of the *presentation*:

- Explanation of your code.
- Proposal of datatype modifications: propose alternative data structures to substitute the ones used in this project (for example, the use of a *nested list* instead of a *dictionary*). Explain the advantages and disadvantages of your proposal.

Presentation: **Nov 30th and Dec 1st, 2026**